Guía del esquemático — placa de nodo DASH

Para dibujarlo en KiCad 8 · versión 2, 24 de octubre de 2026 · Fuente: Documento Maestro v2.2 (11.2, 12, 15.1) y ADR-013 (módulo soldado, ensamblado por el fabricante)

Esta guía te lleva bloque por bloque para dibujar el esquemático de la única placa del sistema: la misma para las 4 puertas, con el rol elegido por jumpers (ADR-009). El objetivo de esta etapa es tener el **esquemático completo y revisado**. El ruteo, la cotización y la fabricación vienen después y esperan a las mediciones de la sección 10.

Cada decisión lleva una etiqueta con su firmeza:

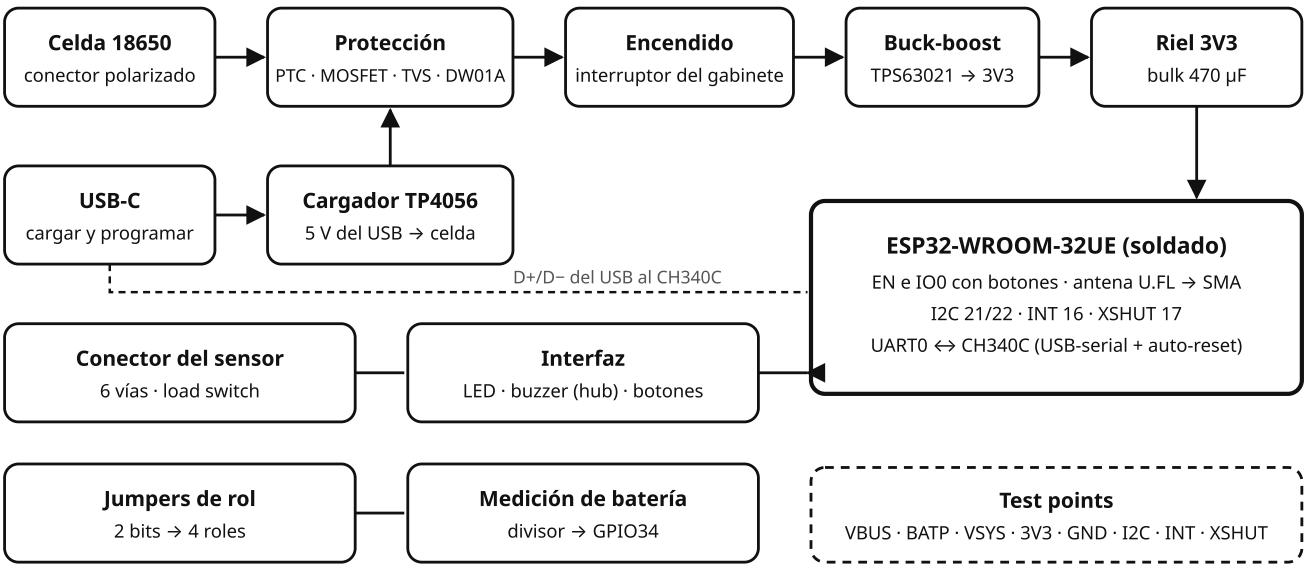
- MAESTRO lo pide el Documento Maestro. No se cambia sin un ADR.
- ADR-013 sale de la decisión de soldar el módulo y mandar a ensamblar (`docs/adr/ADR-013-modulo-soldado-pcba.md`).
- PROPUESTA la propongo yo para completar el diseño. Puedes cambiarla; si la cambias, anótalo.
- PENDIENTE depende de una medición, una hoja de datos o una cotización. Se dibuja, pero no se cierra.

> **Regla de oro para los valores**
>
> Para cada integrado (TP4056, DW01A, TPS63021, CH340C), **copia el circuito típico de su hoja de datos** (la sección "Typical Application"). Los valores que pongo aquí son los habituales para orientarte; si alguno no coincide con la hoja de datos de la pieza que vas a comprar, **manda la hoja de datos**. Para el ESP32, la referencia es el esquemático público de la ESP32-DevKitC de Espressif.

---

1. Qué es esta placa

El **módulo ESP32-WROOM-32UE va soldado** ADR-013. Es la pieza metálica que ya trae el chip, la memoria flash, el cristal y el conector U.FL para la antena externa. Alrededor van: un USB-C para **cargar y programar**, el cargador con protección de la celda, el buck-boost de 3.3 V, el conector del sensor, el LED, el buzzer, los jumpers de rol y los test points. Todo lo SMD en una sola cara, para que el fabricante lo ensamble.



Flechas: camino de la energía. Los 5 V del USB solo alimentan el cargador; el ESP32 siempre se alimenta de la celda a través del buck-boost. La línea punteada son los datos del USB.

---

2. Antes de abrir KiCad

1. **Descarga las hojas de datos**: ESP32-WROOM-32UE, TP4056, DW01A, FS8205A, TPS63021, CH340C y la del VL53L1X (o la de tu breakout). Más el esquemático de la ESP32-DevKitC (Espressif lo publica).
2. **Revisa la huella del WROOM-32UE**: mide 18 × 19.2 mm, más corta que el WROOM-32 normal (18 × 25.5 mm), porque no lleva antena impresa. Si KiCad no trae esa huella exacta, hazla desde la hoja de datos; no uses la del WROOM-32 PENDIENTE.
3. **Busca cada pieza en el catálogo del fabricante** (en JLCPCB: la sección de piezas, que es el inventario de LCSC). Anota su código LCSC y si es "Basic" o "Extended": las Extended cobran una tarifa por cada tipo de pieza distinta. Si alguna no está, elige una equivalente PENDIENTE.
4. Crea el proyecto en `hardware/dash_nodo/` dentro del repo, para que quede versionado.

Organización en KiCad

Hoja	Contenido
1_energia	Batería, protección, interruptor, buck-boost, bulk, medición de batería
2_usb_carga	USB-C, protección ESD, cargador TP4056, CH340C y auto-reset
3_esp32	Módulo, desacoplo, EN e IO0, jumpers de rol
4_sensor	Conector de 6 vías, resistencias serie I2C, load switch, pull-ups opcionales
5_interfaz_prueba	LED, buzzer, botones, test points, notas de serigrafía

Redes con etiquetas globales y nombres fijos: `VBUS`, `VBAT` (celda), `BATP` (tras la protección), `VSYS` (tras el interruptor), `3V3`, `GND` (tierra del sistema), `BAT−` (negativo de la celda), `USB_DP`, `USB_DN`, `TXD0`, `RXD0`, `EN`, `IO0`, `SDA`, `SCL`, `SENS_INT`, `SENS_XSHUT`, `SENS_VIN`, `SENS_EN`, `LED`, `BUZZ`, `ROL0`, `ROL1`, `BTN_A`, `BTN_B`, `VBAT_ADC`.

3. Asignación de pines del ESP32

Los del sensor ya están validados en banco. El resto evita los pines conflictivos: 2, 5, 12 y 15 (deciden el modo de arranque y se dejan **sin conectar**), 6–11 (memoria flash interna del módulo) y los ADC2 (no funcionan con WiFi encendido). **El GPIO12 nunca debe tener pull-up**: si arranca en alto, el módulo configura mal el voltaje de la flash y no arranca.

GPIO	Red	Dirección	Notas	
EN	EN	Entrada	Reset. 10 kΩ a 3V3 + 1 µF a GND + botón RESET a GND + auto-reset	ADR-013
0	IO0	Entrada	Arranque. 10 kΩ a 3V3 + botón BOOT a GND + auto-reset	ADR-013
1 / 3	TXD0 / RXD0	UART0	Al CH340C (cruzados: TX con RX)	ADR-013
21	SDA	E/S	I2C, 100 kHz validado en protoboard	MAESTRO
22	SCL	Salida	I2C	MAESTRO
16	SENS_INT	Entrada	INT del VL53L1X: activo en bajo y latcheado	MAESTRO
17	SENS_XSHUT	Salida OD	Se suelta en alta impedancia, nunca se fuerza a alto	MAESTRO
27	SENS_EN	Salida	Load switch del sensor (en bajo = sensor encendido)	PROPUESTA
25	LED	Salida	LED indicador con 100 Ω	PROPUESTA
26	BUZZ	Salida PWM	Buzzer pasivo vía transistor; sin poblar en nodos	PROPUESTA
32 / 33	ROL0 / ROL1	Entrada	Jumpers a GND; pull-up interno	PROPUESTA
13 / 19	BTN_A / BTN_B	Entrada	Botones a GND; sin poblar en nodos	PROPUESTA
34	VBAT_ADC	Analógica	ADC1 (funciona con WiFi). Solo entrada	PROPUESTA

Roles con 2 jumpers PROPUESTA: interruptor cerrado = 0, abierto = 1. 00 = inicio (0 yd), 01 = split 10 yd, 10 = split 20 yd, 11 = meta + hub (40 yd; con todo abierto, porque es la unidad que más se usa sola en banco). Para que el fabricante lo ensamble, usa un DIP switch SMD de 2 posiciones en lugar de headers con jumper.

4. Energía (hoja 1_energia)

El orden, desde la celda: conector → protección → (aquí se conecta el cargador) → interruptor → buck-boost → 3V3.

1. **Conector de batería** MAESTRO: JST-PH de 2 pines SMD, polarizado. Redes `VBAT` y `BAT-`.
2. **MOSFET-P de polaridad inversa** MAESTRO: drain a `VBAT`, source hacia la carga y gate a `BAT-` con 100 kΩ. De umbral bajo (tipo AO3401): a 3.0 V de celda debe conducir bien.
3. **Fusible PTC** en serie MAESTRO, de sostén ~1.5 A PROPUESTA, con margen sobre los picos del radio. Se ajusta con la medición de la sección 8. Salida: `BATP`.
4. **TVS** entre `BATP` y `BAT-` MAESTRO: unidireccional de 5 V (tipo SMAJ5.0A).
5. **Protección de la celda**: DW01A + FS8205A (dos MOSFET-N en un encapsulado) en el **lado negativo**, entre `BAT-` y `GND` ADR-013. Circuito típico: VCC del DW01A a `BATP` por 100 Ω con 100 nF a `BAT-`; OD y OC a las dos compuertas del FS8205A; CS a `GND` por 1–2 kΩ. Corta la celda por sobrecarga, sobredescarga y cortocircuito.
6. **Interruptor** MAESTRO: conector JST de 2 pines hacia el interruptor del gabinete, entre `BATP` y `VSYS`. El cargador queda **antes** del interruptor, así que la celda se carga con el sistema apagado. Eso evita el problema de cargar con el sistema encendido (sección 5).
7. **Buck-boost TPS63021** (salida fija de 3.3 V; el TPS63020 es la versión ajustable) ADR-013: `VSYS` → `3V3`. Del circuito típico: inductor de 1.5 µH con corriente de saturación ≥ 3 A, capacitores cerámicos a la entrada y a la salida, EN a VIN y FB a VOUT. Para PS/SYNC, elige el nivel que **desactiva el modo de ahorro**: menos rizo con los picos del radio a cambio de unos mA más. Verifica el nivel en la hoja de datos PROPUESTA.
8. **Bulk de 470 µF** en `3V3`, junto al pin de alimentación del módulo MAESTRO. Polímero o tántalo SMD. Su valor final sale de la sección 8 PENDIENTE.
9. **Medición de batería** PROPUESTA: divisor de 100 kΩ / 100 kΩ desde `VSYS` a `VBAT_ADC` (GPIO34), con 100 nF a GND. 4.2 V quedan en 2.1 V. Va después del interruptor para que no descargue la celda con el sistema apagado.

> **La tierra del sistema es la de después del DW01A**
>
> El FS8205A corta la celda por el negativo. Por eso `BAT-` (la celda) y `GND` (todo lo demás) **son redes distintas**, unidas solo a través de los MOSFET. Todo el circuito, incluido el cargador, usa `GND`. Solo el conector de batería, el gate del MOSFET de polaridad, el TVS y el DW01A tocan `BAT-`. Si los unes, la protección deja de funcionar sin que nada lo avise.

5. USB, carga y programación (hoja 2_usb_carga)

1. **USB-C de 16 pines** (solo USB 2.0) ADR-013. CC1 y CC2 cada uno con su propia resistencia de **5.1 kΩ a GND**: sin ellas, un cargador USB-C no entrega 5 V. Une A6 con B6 (D+) y A7 con B7 (D−).
2. **Protección ESD** en D+/D− y VBUS (tipo USBLC6-2SC6) PROPUESTA: el conector queda expuesto en el gabinete.

3. **Cargador TP4056** ADR-013: VCC a `VBUS`, BAT a `BATP` y GND a `GND`. La resistencia de PROG fija la corriente de carga: 2 kΩ ≈ 580 mA PROPUESTA, suave para la celda y con menos calor dentro del gabinete (1.2 kΩ ≈ 1 A). TEMP: a GND si no hay termistor; si quieres el corte por temperatura que sugiere el Maestro 12.4, un NTC de 10 kΩ pegado a la celda PENDIENTE: decisión. CHRG y STDBY a dos LED (rojo y verde) con 1 kΩ desde `VBUS`.
4. **CH340C** (USB-serial, no necesita cristal) ADR-013: VCC y V3 a `3V3` con 100 nF; UD+/UD− a `USB_DP`/`USB_DN`; TXD → `RXD0` y RXD ← `TXD0`. Se alimenta del riel de 3.3 V: **para programar, el interruptor tiene que estar encendido**. A cambio, sus niveles quedan en 3.3 V, que es lo que tolera el ESP32.
5. **Auto-reset** ADR-013: el circuito de dos NPN de la DevKitC. Q1 con base a DTR por 10 kΩ, emisor a RTS y colector a `EN`. Q2 con base a RTS por 10 kΩ, emisor a DTR y colector a `IO0`. Con eso, el IDE y arduino-cli programan sin tocar botones, igual que con tus placas de banco.

Por qué esto resuelve el problema de las dos fuentes

Los 5 V del USB van **solo** al cargador. El ESP32 y el CH340C siempre se alimentan de `3V3`, que solo viene del buck-boost. Nunca hay dos reguladores empujando el mismo riel, que era el problema de la versión con tarjeta enchufada.

Limitación conocida: el TP4056 no separa la carga de la celda del consumo del sistema. Si cargas con el interruptor encendido, la carga puede no terminar bien. Por eso el cargador va antes del interruptor y la regla es: **cargar con el sistema apagado y nunca en campo** (Maestro 12.4). Ponlo en la serigrafía.

6. ESP32 (hoja 3_esp32)

1. **Módulo WROOM-32UE** con todos sus pines de GND y la **almohadilla central** a `GND` (en la PCB, con varias vías al plano).
2. **Desacoplo** en el pin 3V3 del módulo: 10 µF + 100 nF (diseño de Espressif) más el 1 µF que pide el Maestro (MAESTRO), lo más cerca posible del pin.
3. **EN**: 10 kΩ a 3V3, 1 µF a GND (el retardo de arranque que recomienda Espressif) y botón RESET a GND.
4. **IO0**: 10 kΩ a 3V3 y botón BOOT a GND. Los dos botones son de respaldo por si el auto-reset falla; se pueblan en todas las placas.
5. **GPIO 2, 5, 12 y 15**: "no connect". No les pongas nada.
6. **Antena**: el conector U.FL ya viene en el módulo; un cable U.FL a SMA lo lleva a la antena del gabinete. No hay nada que dibujar, pero deja libre el espacio alrededor del conector en la PCB.
7. **Roles**: DIP switch de 2 posiciones entre `ROL0/ROL1` y `GND`.

7. Sensor, interfaz y prueba (hojas 4 y 5)

Sensor

1. **Conector JST-PH de 6 vías** (MAESTRO): 1 `SENS_VIN`, 2 `GND`, 3 `SDA`, 4 `SCL`, 5 `SENS_INT`, 6 `SENS_XSHUT`.
2. **33 Ω en serie** en SDA y SCL, del lado del ESP32 (MAESTRO).
3. **Load switch** (MAESTRO): MOSFET-P (AO3401) con source en `3V3`, drain en `SENS_VIN`, gate a `SENS_EN` (GPIO27) y 10 kΩ de gate a `3V3` (PROPUESTA): mientras el ESP32 arranca, el sensor queda apagado. Pon 1 µF en `SENS_VIN`.
4. **Pull-ups opcionales**, marcados como DNP (no poblar) (PENDIENTE): 4.7 kΩ de SDA y SCL y 10 kΩ de `SENS_INT`, todos a `SENS_VIN`, no a 3V3. Con el sensor apagado, un pull-up a 3V3 lo alimentaría "por detrás" a través de sus pines. Solo se pueblan si el breakout no los trae (Maestro 12.2).
5. `SENS_XSHUT` directo al GPIO17, sin pull-up propio (MAESTRO).

Interfaz

- **LED**: GPIO25 → 100 Ω → LED → GND (MAESTRO).
- **Buzzer pasivo** (DNP salvo en el hub) (MAESTRO): GPIO26 → 1 kΩ → base de un NPN. Buzzer entre 3V3 y el colector, con diodo de rueda libre si es magnético.
- **Botones** (DNP en nodos) (MAESTRO): GPIO13 y GPIO19 a GND, con 100 nF en paralelo.

Prueba y serigrafía

- **Test points** (MAESTRO): `3V3`, `GND`, `SDA`, `SCL`, `SENS_INT`, `SENS_XSHUT`. Agrega `VBUS`, `BATP` y `VSYS`, y al menos dos de GND junto a los de energía (PROPUESTA). Usa pads SMD, no pines: no cuestan ensamblado.
- **Puentes de 0 Ω** en `VSYS` (antes del buck-boost) y en `3V3` (antes del módulo) (PROPUESTA). En la primera energización de cada placa se prueba cada bloque por separado: si el buck-boost sale

mal, no se lleva al ESP32 de corbata. Es la mitigación principal del riesgo de no poder rehacer la placa (ADR-013).

- **Serigrafía** MAESTRO: proyecto, versión y fecha; función y pin 1 de cada conector; tabla de roles; autor y URL del repositorio; "Cargar apagado".

---

8. Medición del riel de 3.3 V con el osciloscopio

El hub se reinició 422 veces por brownout en el Experimento 0b, con el SoftAP cargado. Esta medición dice cuánto cae el riel de 3.3 V en los picos del radio. Con eso se fijan el bulk, el desacoplo y el PTC. Se puede hacer ya con el hub de banco.

Punto	Dónde	Por qué
3V3	Pin 3V3 de la tarjeta, con la tierra de la punta en el GND más cercano (clip corto o resorte)	Es lo que ve el ESP32; su mínimo de operación es 3.0 V
5V	Pin 5V de la tarjeta	Si cae aquí, el problema es el cable o el puerto USB

1. Hub del 0b en **modo ráfagas con SoftAP** y la carga de S3 (1 teléfono en estrés y 2 escuchando). El runner flashea y tú conectas los teléfonos.
2. Punta en ×10, acoplamiento **DC**, 0.5 V/div, trazo centrado en ~3.3 V.
3. Base de tiempo de 50–100 µs/div para ver un pico; luego 1–2 ms/div para ver varios.
4. **Disparo por flanco de bajada** en ~3.1 V, modo normal. Congela la peor caída.
5. Si tu equipo lo tiene: medición automática de Vmin o modo persistencia durante 1–2 min.

#	Alimentación	Carga	Anotar
1	USB de la laptop	SoftAP apagado	Vmin en 3V3 y 5V (referencia)
2	USB de la laptop	S3	Vmin, duración de la caída, reinicios
3	Cargador de pared 5 V / 2 A, cable corto	S3	Si mejora, el brownout era de la fuente
4	Celda + módulo TPS63020 de banco directo a 3V3, sin USB	S3	La condición real de la placa

Criterio PROPUESTA: Vmin ≥ 3.1 V en la corrida 4. Antes de medir, escribe tu predicción (o que no la haces) en `docs/ESTADO.md`.

---

9. Paso a paso en KiCad

☐ Crear `hardware/dash_nodo/`, llenar el cajetín (título, versión **v0.1**, fecha, autor).

☐ Crear las 5 hojas jerárquicas.

☐ Librería del proyecto `dash.kicad_sym`: los símbolos que no estén en KiCad (DW01A, FS8205A, TPS63021, CH340C si faltan) y la huella del WROOM-32UE.

▫ Agregar a cada componente un campo **LCSC** con su código; el fabricante lo usa para el ensamblado.

▫ Dibujar las hojas 1 y 2 copiando los circuitos típicos de las hojas de datos.

▫ Dibujar las hojas 3, 4 y 5.

▫ **PWR_FLAG** en `VBUS`, `VBAT`, `GND`, `BAT-` y `3V3`.

▫ "No connect" en GPIO 2, 5, 12, 15 y en los pines del módulo que no se usan.

▫ Anotar designadores y poner el **Valor** de cada componente.

▫ Marcar **DNP**: pull-ups opcionales, buzzer y botones de interfaz.

▫ **ERC** en cero errores; las advertencias que aceptes, con una nota de texto.

▫ Exportar el esquemático a PDF y la BOM a CSV en `hardware/`, y hacer commit (o pedírmelo).

Cómo lo revisamos

- **ERC** sin errores; cada GPIO de la sección 3, una sola vez y en su red.
- **VBUS** solo llega al cargador, al ESD y a los LED de carga; nunca a 3V3 ni a VSYS.
- **BAT- y GND** son redes distintas, unidas solo por el FS8205A.
- **GPIO12** sin nada conectado; EN con su RC; IO0 con pull-up.
- **XSHUT** sin pull-up propio; los pull-ups opcionales van a SENS_VIN.
- **Cada integrado** tiene su circuito típico completo, y cada pieza su código LCSC.

Cuando lo termines, pásame el PDF exportado o el `.kicad_sch` y lo reviso contra esta lista y contra el Maestro.

---

10. Qué falta antes de rutear y fabricar

Pendiente	Qué cambia	Cómo se cierra
Medición del riel de 3.3 V (sección 8)	Bulk, desacoplo, PTC	Corridas 1–4 con el osciloscopio
Disponibilidad y costo en el catálogo del fabricante	Piezas equivalentes, presupuesto (Maestro 15.2)	Códigos LCSC y cotización de 5 placas ensambladas
Huella del WROOM-32UE	Footprint	Desde su hoja de datos
Pull-ups de I2C e INT, I2C a 400 kHz	Poblar o no las resistencias DNP	Revisar el breakout y probar 400 kHz
Termistor de la celda (TP4056 TEMP)	NTC o TEMP a GND	Decisión tuya
Experimentos 2 y 3	Posición de conectores, gabinete, ventana	Plan de validación (Maestro 16)